

Computationally Efficient Algorithm to Generate a Waypoints-Based Trajectory for a Quadrotor UAV

Ghulam Farid¹, Hongwei Mo¹, Muhammad Irshad Zahoor², and Qu Liwei¹

1. College of Automation, Harbin Engineering University, Harbin 150001, China

E-mail: farid.anjum@yahoo.com

2. College of Information and Communication Engineering, Harbin Engineering University, Harbin 150001, China

Abstract: The complex dynamical burden of a quadrotor unmanned aerial vehicle (UAV) can be reduced by designing dynamically feasible and computationally fast trajectories. This brief discusses the waypoints-based trajectory generation algorithm specifically for a quadrotor UAV and considers linear interpolation along with parabolic blends. The aforementioned approach has only been studied for robot manipulators in the literature, on the other hand, it can also be used to design smooth trajectories for underactuated systems like quadrotors. The linear interpolation is computationally fast as compared to the polynomial based interpolation schemes, however, the former approach results in discontinuities at waypoints which can be eradicated using parabolic blends. Furthermore, flatness theory is conferred which helps in designing the trajectories in flat output space. A nonlinear trajectory tracking controller is also designed to test the generated trajectories in simulation environment and presented results infer smooth tracking control.

Key Words: Quadrotor UAV, Trajectory Generation, Interpolation, Parabolic Blends

1 INTRODUCTION

In the past decade, extensive work has been reported on unmanned aerial vehicles (UAVs), with special focus on the quadrotor type of multi-rotor UAVs [1-5]. In particular, recent advances encompass control design and planning issues [6-10]. Most of the planning issues involve the generation of dynamically feasible and smooth trajectories [11, 12]. Smooth trajectories reduce the burden from UAV dynamics and its controller by limiting the magnitude of undesirable vibrations which may occur due to the rough and non-smooth trajectories. Therefore, the controller performance is closely related to the desired path, generated for a particular mission which ultimately makes the trajectory planning an important issue in aerial robotics [13].

Generating the trajectories using time varying continuous functions is rather difficult for a UAV user. Instead, intuitive methods can be employed which accept user defined waypoints as arguments and generate the trajectory online by the help of predefined program which interpolates using different ordered polynomial functions. This enormously diminishes the burden from user by offering a way of just defining spatial points.

Most of the trajectory planning issues have been solved in the literature using various kinds of splines and polynomials in order to interpolate between the waypoints [14, 15]. These higher ordered polynomials pose higher computational costs due to complex calculation structure for coefficients. In addition, Runge's phenomenon become more apparent as one goes for higher order polynomials.

The simplistic way of interpolation is the linear interpolation and a trivial trajectory can be constructed which takes less computational time and stays within a specified linear corridor. However, the aforesaid approach requires the quadrotor to come to rest at waypoints due to infinite curvatures [16]. This is obviously not a choice to be appreciated as the UAV needs to have continuous motion at curvature points with specified velocities and acceleration profiles. This problem can be solved using parabolic blends at waypoints to make the curvatures finite and smooth. Therefore, the overall methodology offers a computationally fast algorithm which generates feasible and smooth trajectories. Even though, this methodology has been used for several robotic manipulator applications, however, it has never been implemented to generate the trajectories for a quadrotor UAV. The contribution of this paper is the mathematical design and procedure of generating the trajectories for a quadrotor UAV using linear interpolation and parabolic blend strategies. Apart from trajectory generation algorithm, this paper also discusses modeling and control design. The controller is designed using inverse dynamics based control method i.e. feedback linearization method, which eventually is used to track the generated trajectories. Furthermore, differential flatness is discussed briefly in order to convey the idea that quadrotor dynamics are flat and inputs along with system states can be expressed as a function of carefully selected flat outputs. This helps in designing the trajectory in flat output space which ultimately, is the function of quadrotor inputs and dynamic states.

The rest of the paper is organized as follows; section 2 describes modeling, control and differential flatness.

This work is supported by Chinese Government Scholarship Council.

Section 3 presents complete mathematical flow of algorithm for interpolation and trajectory generation. In section 4, results of numerical simulations are discussed. Finally, section 5 closes the paper with conclusions.

2 PRELEMINARIES

2.1 Modeling

Referring to Fig. 1, there are two reference frames, one is body fixed frame B and the other is Earth reference frame E (i.e. world reference frame). Transformation of quadrotor dynamics from body to world reference frame can be accomplished using rotation matrix R . The Euler angles roll, pitch, and yaw are symbolized by $[\phi \ \theta \ \psi]^T$ respectively. The body angular rates are represented as $\omega_B = [p \ q \ r]^T$. The position ζ and velocity V state equations can be written in world reference frame as:

$$\frac{d}{dt} \zeta = V \quad (1)$$

$$\frac{d}{dt} V = -g e_z + \frac{F_B}{m} R e_z \quad (2)$$

$$\frac{d}{dt} R = R S(\omega_B) \quad (3)$$

where $S(\omega_B)$ is a skew symmetric matrix.

$$S(\omega_B) = \begin{bmatrix} 0 & -r & q \\ r & 0 & -p \\ -q & p & 0 \end{bmatrix} \quad (4)$$

There are two major dynamic entities produced by the rotation of the rotor, one is lift force or thrust and the other is moment around the spinning axes. Thrust owns a key role in all types of quadrotor maneuvers, hence an important term in control design process. The rotation of the rotors defines following four control terms:

$$\left. \begin{aligned} F_B &= b(\Omega_1^2 + \Omega_2^2 + \Omega_3^2 + \Omega_4^2) \\ \tau_{B\phi} &= lb(\Omega_4^2 - \Omega_2^2) \\ \tau_{B\theta} &= lb(\Omega_3^2 - \Omega_1^2) \\ \tau_{B\psi} &= d(\Omega_4^2 + \Omega_2^2 - \Omega_3^2 - \Omega_1^2) \end{aligned} \right\} \quad (5)$$

where b and d represent thrust and drag factors respectively, while Ω_i represents propeller speed.

By letting F_B equal to U_1 and torque vector τ_B equal to $[U_2 \ U_3 \ U_4]^T$, we can extract following mathematical model in world reference frame from (1-5).

$$\left. \begin{aligned} \ddot{x} &= (\sin\phi\sin\psi + \cos\phi\sin\theta\cos\psi) \frac{1}{m} U_1 \\ \ddot{y} &= (\cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi) \frac{1}{m} U_1 \\ \ddot{z} &= -g + (\cos\phi\cos\theta) \frac{1}{m} U_1 \end{aligned} \right\} \quad (6)$$

$$\left. \begin{aligned} \ddot{\phi} &= \dot{\theta}\dot{\psi} \left(\frac{I_y - I_z}{I_x} \right) - \frac{I_r}{I_x} \dot{\theta}\Omega_r + \frac{1}{I_x} U_2 \\ \ddot{\theta} &= \dot{\phi}\dot{\psi} \left(\frac{I_z - I_x}{I_y} \right) + \frac{I_r}{I_y} \dot{\phi}\Omega_r + \frac{1}{I_y} U_3 \\ \ddot{\psi} &= \dot{\phi}\dot{\theta} \left(\frac{I_x - I_y}{I_z} \right) + \frac{1}{I_z} U_4 \end{aligned} \right\} \quad (7)$$

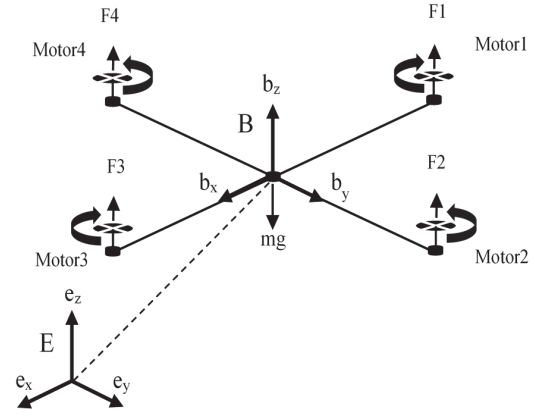


Fig. 1. Quadrotor configuration with reference frames

2.2 Control

This section deals with controller design using inverse dynamics based control approach. Specifically, this methodology helps in removing nonlinearities from complex nonlinear plant models, i.e., like quadrotor UAV. Numerous studies have conferred this approach to design a flight controller for aerial unmanned vehicles. The key advantage of this approach is the generation of linearized plant model by the cancelation of nonlinear terms. Therefore, simple classical linear control theory can be applied on resulting UAV model to design a flight controller. To follow the desired trajectories, controller commands are computed in this section.

Following equation can be derived from (6) in order to calculate the desired thrust:

$$U_{1d} = m[\cos\phi\cos\theta]^T (K_p z_e + K_d \dot{z}_e + g) \quad (8)$$

where $z_e = z_d - z$ is error, K_p, K_d are tuning gains.

To develop the formalism between (6) and (7), linearization supposition is made around hover state. This makes the sense that thrust input U_1 will be equal to mg for (6), and letting roll and pitch angles equal to zero, we have:

$$\ddot{x} = g(\Delta\phi\sin\psi + \Delta\theta\cos\psi) \quad (9)$$

$$\ddot{y} = g(\Delta\theta\sin\psi - \Delta\phi\cos\psi) \quad (10)$$

This generates:

$$\phi_d = \frac{1}{g} (\ddot{x}\sin\psi - \ddot{y}\cos\psi) \quad (11)$$

$$\theta_d = \frac{1}{g} (\ddot{x}\cos\psi + \ddot{y}\sin\psi) \quad (12)$$

The equations $\ddot{x} = \ddot{x}_d + K_d \dot{e} + K_p e$ and $\ddot{y} = \ddot{y}_d + K_d \dot{e} + K_p e$ will force the system to follow desired Cartesian commands in x-y plane. The controller equations to control the torque entities can be extracted from (7) as:

$$\left. \begin{aligned} U_{2d} &= \frac{1}{I_x} (I_x(K_p \phi_e + K_d \dot{\phi}_e) + I_r \dot{\theta}\Omega_r - \dot{\theta}\dot{\psi}(I_y - I_z)) \\ U_{3d} &= \frac{1}{I_y} (I_y(K_p \theta_e + K_d \dot{\theta}_e) - I_r \dot{\phi}\Omega_r - \dot{\phi}\dot{\psi}(I_z - I_x)) \\ U_{4d} &= I_z(K_p \psi_e + K_d \dot{\psi}_e) - \dot{\phi}\dot{\theta}(I_x - I_y) \end{aligned} \right\} \quad (13)$$

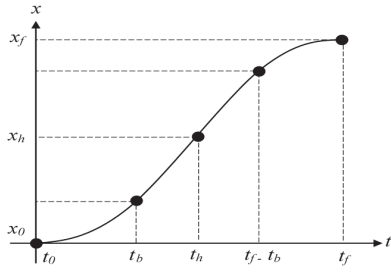


Fig. 2. Linear interpolation with parabolic blends (two points case)

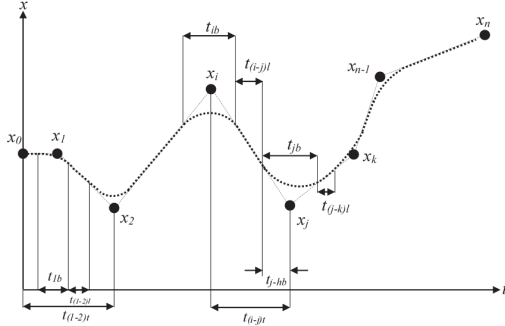


Fig. 3. Linear interpolation with parabolic blends (multiple points case)

where $\phi_e = \phi_d - \phi$ and $\theta_e = \theta_d - \theta$.

Following relation can be used to compute the desired motor velocities from desired controller commands.

$$\begin{bmatrix} \Omega_{1d}^2 \\ \Omega_{2d}^2 \\ \Omega_{3d}^2 \\ \Omega_{4d}^2 \end{bmatrix} = \begin{bmatrix} b & b & b & b \\ 0 & -b & 0 & b \\ -b & 0 & b & 0 \\ -d & d & -d & d \end{bmatrix}^{-1} \begin{bmatrix} U_{1d} \\ U_{2d} \\ U_{3d} \\ U_{4d} \end{bmatrix} \quad (14)$$

where $\Omega_{id}^2, i = 1, 2, 3, 4$ symbolizes desired commands to regulate the motor speeds.

2.3 Differential Flatness

The quadrotor underactuated dynamics are differentially flat. According to flatness theory, dynamics of a complex nonlinear system are differentially flat if the inputs and states of the system can be expressed in terms of flat outputs and their successive derivatives. Fortunately, quadrotor UAV dynamics are differentially flat and detailed proof is conferred in [16]. Flatness property of quadrotor helps in generating trajectories automatically since any smooth function of trajectories, which is described in flat output space, can be followed by a quadrotor UAV.

The selected flat outputs of the quadrotor UAV are:

$$\chi = [x, y, z, \psi]^T \rightarrow \mathbb{R}^3 \times SO(2) \quad (15)$$

3 INTERPOLATION AND TRAJECTORY GENERATION

A simplistic way that can be used to generate a smooth trajectory for a quadrotor UAV, is a linear interpolation between the waypoints with parabolic blends to shape the path. One of the major shortcomings associated with this scheme is that quadrotor must have to stop after reaching a

particular waypoint, this makes the velocity profile discontinuous. For a smooth trajectory, continuous position and velocity profiles are inevitable, therefore, parabolic blends can be added at intermediate waypoints that are mutually connected through linear functions.

3.1 Interpolation Between Two Points

This scenario deals with the basic build up by considering only a single segment which connects two waypoints or via-points. The basic motive is the addition of parabolic blends at both ends of the segment. This requires the assumption that duration of the blends and acceleration profiles are same at each end. Referring to the fig. 2, it is obvious that solution of the problem is symmetric around the halfway point of the segment. It is important to note that, for the first blend the velocity of blend region at exit point must equal the velocity of the linear region. Mathematically, we can formulate this fact as:

$$\ddot{\chi}t_b = \frac{\chi_h - \chi_b}{t_h - t_b} \quad (16)$$

Where, $\ddot{\chi}$ is the value of acceleration during the blend section, χ_b, χ_h represent the values of χ at the end of blend and at end of the linear section respectively, t_b and t_h symbolize the blend time and time at start of next blend region correspondingly. Now χ_b can be calculated as:

$$\chi_b = \chi_0 + \frac{\ddot{\chi}t_b^2}{2} \quad (17)$$

Since $t = 2t_h$, we can get following from (16) and (17):

$$\ddot{\chi}t_b^2 - \ddot{\chi}tt_b + (\chi_f - \chi_0) = 0 \quad (18)$$

Here, t is the total time duration of the segment and is user definable. In above equation, the values χ_f, χ_0 are also user definable while blend time t_b is calculated by choosing arbitrary value of the acceleration $\ddot{\chi}$. The acceleration value must be chosen relatively high in order to retain a sufficient linear portion in the segment path. Following inequality is used as lower bound for acceleration:

$$\ddot{\chi} \geq \frac{4(\chi_f - \chi_0)}{t^2} \quad (19)$$

When the value of acceleration is exactly at lower bound, the linear portion of segment vanishes and segment consists of only the blends. Consequently, by increasing the value of acceleration the length of linear portion increases and blend region decreases. From (18), we can calculate blend time t_b as:

$$t_b = \frac{t}{2} - \sqrt{\frac{\ddot{\chi}^2 t^2 - 4\ddot{\chi}(\chi_f - \chi_0)}{4\ddot{\chi}^2}} \quad (20)$$

3.2 Interpolation Between Multiple Points

To deal with n number of waypoints using linear functions and parabolic blend method, previous calculations can be augmented further. Multiple path segments and waypoints are shown in fig. 3. The initial point on the trajectory path is χ_0 , and final point is χ_n . Whole trajectory with some intermediate path points is $(\chi_0, \chi_1, \chi_2, \dots, \chi_i, \chi_j, \chi_k, \dots, \chi_{n-1}, \chi_n)$. First, consider three adjacent path points (χ_i, χ_j, χ_k) , the blend time for χ_i is t_{ib} and $t_{(i-j)}$

represents the time duration of linear region between points χ_i and χ_j . The total time duration from χ_i to χ_j is denoted by $t_{(i-j)t}$. The slope of linear portion is represented by $\dot{\chi}_{i-j}$, which is also known as velocity of linear region and the acceleration at point χ_i is termed as $\ddot{\chi}_i$. It is important to note that the acceleration $\ddot{\chi}_i$ and total time $t_{(i-j)t}$, both are user definable.

The blend time t_{jb} for χ_j can be calculated as:

$$\dot{\chi}_{i-j} = \frac{\chi_j - \chi_i}{t_{(i-j)t}} \quad (21)$$

$$\ddot{\chi}_j = \text{sgn}(\dot{\chi}_{j-k} - \dot{\chi}_{i-j})|\ddot{\chi}_j| \quad (22)$$

$$t_{jb} = \frac{\dot{\chi}_{j-k} - \dot{\chi}_{i-j}}{\ddot{\chi}_j} \quad (23)$$

The linear region duration can be calculated as:

$$t_{(i-j)l} = t_{(i-j)t} - \frac{1}{2}(t_{ib} + t_{jb}) \quad (24)$$

Similar calculations can be performed for all other intermediate waypoints except to the first and last points. Latter scenario follows a slightly different calculation procedure because whole blend time must be considered in total segment duration.

Referring to the fig. 3, for linear region between point χ_1 and χ_2 we have got two expressions for velocity profile or slope of the region. Equating both we have:

$$\frac{\chi_2 - \chi_1}{t_{(1-2)t} - t_{1/2}} = \ddot{\chi}_1 t_1 \quad (25)$$

Equation (25) can be solved to calculate the blend time t_{1b} for χ_1 . This follows as:

$$\ddot{\chi}_1 = \text{sgn}(\chi_2 - \chi_1)|\ddot{\chi}_1| \quad (26)$$

$$t_{1b} = t_{(1-2)t} - \sqrt{t_{(1-2)t}^2 - 2\Delta\chi_{2-1}/\ddot{\chi}_1} \quad (27)$$

While $\Delta\chi_{2-1} = \chi_2 - \chi_1$ and $t_{(1-2)t}$ is the total time duration of the segment between points χ_1 and χ_2 . The linear region duration can be calculated as:

$$t_{(1-2)l} = t_{(1-2)t} - t_1 - \frac{1}{2}t_2 \quad (28)$$

Similarly, for the last segment between points χ_{n-1} and χ_n .

$$\frac{\chi_n - \chi_{n-1}}{t_{(n-1,n)t} - t_{n/2}} = \ddot{\chi}_n t_n \quad (29)$$

$$\ddot{\chi}_n = \text{sgn}(\chi_n - \chi_{n-1})|\ddot{\chi}_n| \quad (30)$$

Now the blend time for n^{th} point can be calculated as:

$$t_{nb} = t_{(n-1,n)t} - \sqrt{t_{(n-1,n)t}^2 + 2\Delta\chi_{(n-1,n)}/\ddot{\chi}_1} \quad (31)$$

And the linear region duration is:

$$t_{(n-1,n)l} = t_{(n-1,n)t} - t_n - \frac{1}{2}t_{n-1} \quad (32)$$

The above presented procedure can be used to find the blend times and the velocity profiles for the segments that connect through waypoints. The user has to specify the waypoints, time duration and acceleration only. On the contrary, optimal time and acceleration profiles can also be assigned

by the system using some heuristic algorithms. The convenient choice is the usage of larger acceleration values at all blends in order to have enough time to stay in the linear portion before the next blend starts.

The calculations performed previously, generate a set of data for each intermediate segment and can be further used by the trajectory generation algorithm to calculate χ , $\dot{\chi}$, and $\ddot{\chi}$. It is pertinent to get the update of time t in order to determine that the system is residing in linear or blend portion of the segment.

For linear portion the trajectory is generated as:

$$\chi = \chi_i + \dot{\chi}_{i-j}t \quad (33)$$

$$\dot{\chi} = \dot{\chi}_{i-j} \quad (34)$$

$$\ddot{\chi} = 0 \quad (35)$$

where t represents the time that has been elapsed since the system calculated i^{th} waypoint and $\dot{\chi}_{i-j}$.

For blend region, let's define half blend time t_{hb} as:

$$t_{hb} = t - (\frac{1}{2}t_i + t_{i-j}) \quad (36)$$

Finally, the trajectory can be generated as follows:

$$\chi = \chi_i + \dot{\chi}_{i-j}(t - t_{hb}) + \frac{1}{2}\ddot{\chi}_j t_{hb}^2 \quad (37)$$

$$\dot{\chi} = \dot{\chi}_{i-j} + \ddot{\chi}_j t_{hb} \quad (38)$$

$$\ddot{\chi} = \ddot{\chi}_j \quad (39)$$

4 NUMERICAL SIMULATIONS

Mathematical design procedure presented in section 3 was implemented in MATLAB environment to generate the trajectories for quadrotor UAV. The inverse dynamics-based controller conferred earlier was used to track these trajectories. Fig. 4 illustrates the linear interpolation scheme for single segment with different parabolic blend regions. These regions are controlled using inequality (19). Multi-segment approach is illustrated in fig. 5, in which initially the trivial trajectory was constructed using only the linear interpolation in left half of the figure, then parabolic blends were added and are shown in right half of the figure. Fig. 6 shows the deteriorated performance of designed controller against non-smooth trivial trajectory.

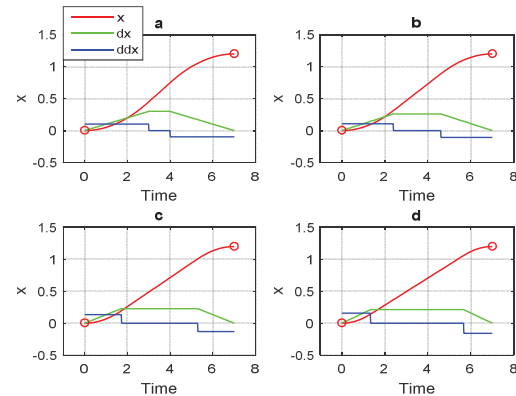


Fig. 4. Linear interpolation with various parabolic blend durations

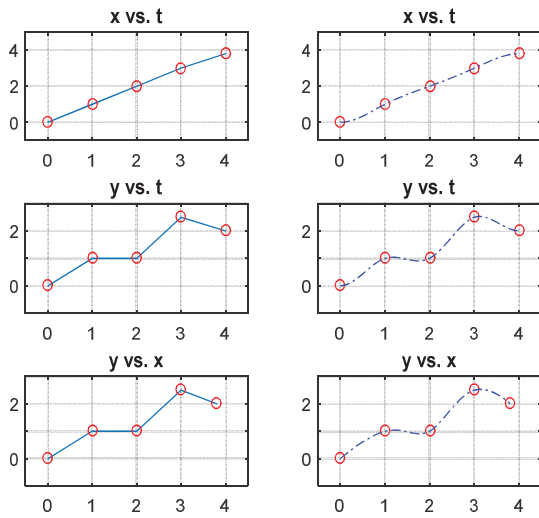


Fig. 5. Multi-segment approach (two-dimensional)

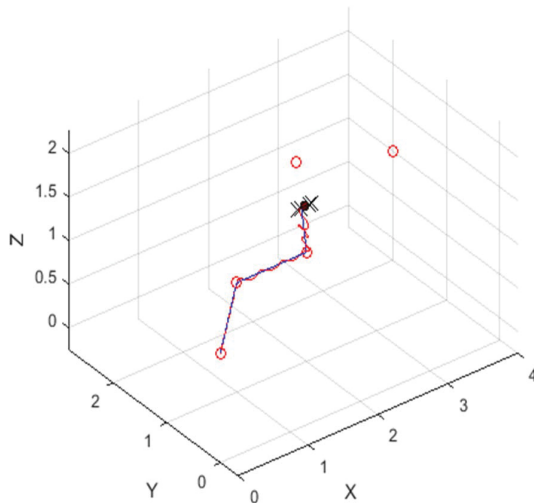


Fig. 6. Deteriorated controller performance against a trivial trajectory (linear interpolation with infinite curvatures)

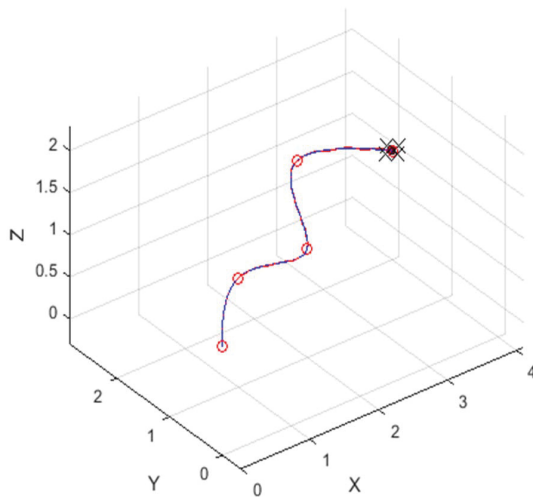


Fig. 7. Generation of three-dimensional smooth trajectory using presented scheme and faithful tracking by the quadrotor

Referring to fig. 7, a smooth trajectory that has been generated using the linear interpolation and parabolic blends, is tracked faithfully by the flight controller. At waypoints, the curvatures are finite and quadrotor can easily traverse the path with desired velocity and acceleration profiles.

5 CONCLUSIONS

Computationally fast trajectory generation algorithm has been designed for a quadrotor UAV. Initially trivial trajectory is generated using linear interpolation which gives fast calculation response, then parabolic blends are added at the knot points to make the curvatures finite and smooth. Differential flatness theory is discussed to design the trajectory in flat output space of the quadrotor dynamics. Furthermore, a nonlinear flight controller is also designed to track the generated trajectories and controller performance is illustrated against smooth and non-smooth trajectories.

REFERENCES

- [1] A. Benallegue, A. Mokhtari, and L. Fridman, High-order sliding-mode observer for a quadrotor UAV, *International Journal of Robust and Nonlinear Control*, vol. 18, 2008, 427-440.
- [2] L. Besnard, Y. B. Shtessel, and B. Landrum, Quadrotor vehicle control via sliding mode controller driven by sliding mode disturbance observer, *Journal of the Franklin Institute*, vol. 349, 2012, 658-684.
- [3] F. Chen, *et al.*, A novel nonlinear resilient control for a quadrotor UAV via backstepping control and nonlinear disturbance observer, *Nonlinear Dynamics*, vol. 85, 2016, 1281-1295.
- [4] G. Farid, *et al.*, A review on linear and nonlinear control techniques for position and attitude control of a quadrotor, *Control and Intelligent Systems*, vol. 45, 2017.
- [5] H. Mo and G. Farid, Nonlinear and adaptive intelligent control techniques for quadrotor UAV- a survey, *Asian Journal of Control*, vol. 21, 2019, 1-20.
- [6] F. Kendoul, Survey of advances in guidance, navigation, and control of unmanned rotorcraft systems, *Journal of Field Robotics*, vol. 29, 2012, 315-378.
- [7] M. H. Jaffery, *et al.*, Experimental quadrotor flight performance using computationally efficient and recursively feasible linear model predictive control, *International Journal of Control*, vol. 86, 2013/12/01 2013, 2189-2202.
- [8] S. Islam, P. X. Liu, and A. E. Saddik, Robust Control of Four-Rotor Unmanned Aerial Vehicle With Disturbance Uncertainty, *IEEE Transactions on Industrial Electronics*, vol. 62, 2015, 1563-1571.
- [9] J.-J. Xiong and E.-H. Zheng, Position and attitude tracking control for a quadrotor UAV, *ISA Transactions*, vol. 53, 2014, 725-731.
- [10] G. Farid, *et al.*, On control law partitioning for nonlinear control of a quadrotor UAV, *2018 15th International Bhurban Conference on Applied Sciences and Technology (IBCAST)*, 2018, 257-262.
- [11] R. Mahony, V. Kumar, and P. Corke, Multirotor Aerial Vehicles: Modeling, Estimation, and Control of

- Quadrotor, *IEEE Robotics & Automation Magazine*, vol. 19, 2012, 20-32.
- [12] D. Mellinger, N. Michael, and V. Kumar, Trajectory generation and control for precise aggressive maneuvers with quadrotors, *The International Journal of Robotics Research*, vol. 31, 2012, 664-674.
- [13] P. Castillo-García, L. E. Muñoz Hernandez, and P. García Gil, "Chapter 9 - Trajectory Generation, Planning & Tracking*," in *Indoor Navigation Strategies for Aerial Autonomous Systems*, ed: Butterworth-Heinemann, 2017, pp. 213-241.
- [14] V. Roldão, *et al.*, A leader-following trajectory generator with application to quadrotor formation flight, *Robotics and Autonomous Systems*, vol. 62, 2014/10/01/ 2014, 1597-1609.
- [15] S. K. Phang, *et al.*, Systems design and implementation with jerk-optimized trajectory generation for UAV calligraphy, *Mechatronics*, vol. 30, 2015/09/01/ 2015, 65-75.
- [16] D. Mellinger and V. Kumar, Minimum snap trajectory generation and control for quadrotors, presented at the Robotics and Automation (ICRA), 2011 IEEE International Conference on. IEEE, 2011.